ID: 22299480

SRS (Software Requirement Specification) document

Project title: **Emporio**

Author name: Mihir Das

Document version: Final

Body

1. Introduction (type of project, purpose, target users):

It is a full stack web application which is made with MERN stack (mongo.db, express.js, react.js and node.js) where react.js is used for frontend and the rest is used for backend development.

"**Emporio**" is a digital product marketplace which will act as a meeting point for designers, developers, writers etc. It is designed to put digital assets like e-books, templates, code snippets, icons, photos, Notion templates etc**.** under a hood so that users can find their necessary needs in one place and reduce the amount of hassle. There will also be a community page where users can communicate with each other via posting. So, the main purpose of this project is to bring all the things in one place so that it becomes easy for users.

Who is it for:

- Designers, Developers, Writers
- Students and in general all people
- Those who are just starting and those who are already are experienced in their works

2. Functional requirements:

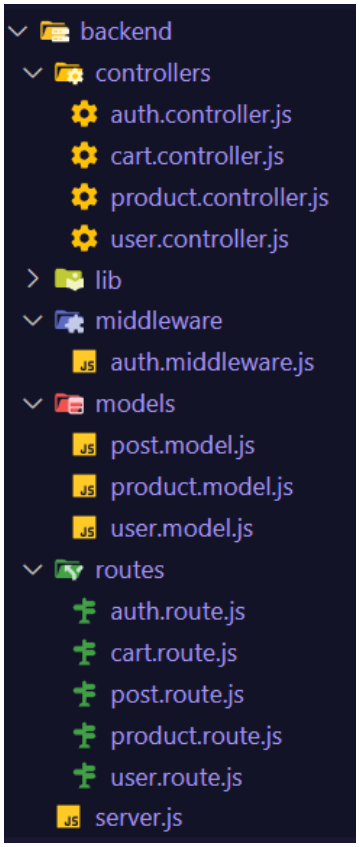
1) Product Management
2) Search by name, category, description
3) User Reviews & Ratings
4) Wishlist System via voting
5) Community - discussion
6) User Profile
7) Admin Dashboard
8) Analytics
9) Announcement System
10) Cart System

3. **Non-functional requirements:**

- Scalability
- Usability

Sprint 1

No. of features completed: 2 (Product Cards and Admin Dashboard)
Excluded from features but done: (Added signup, login, logout with authentication)

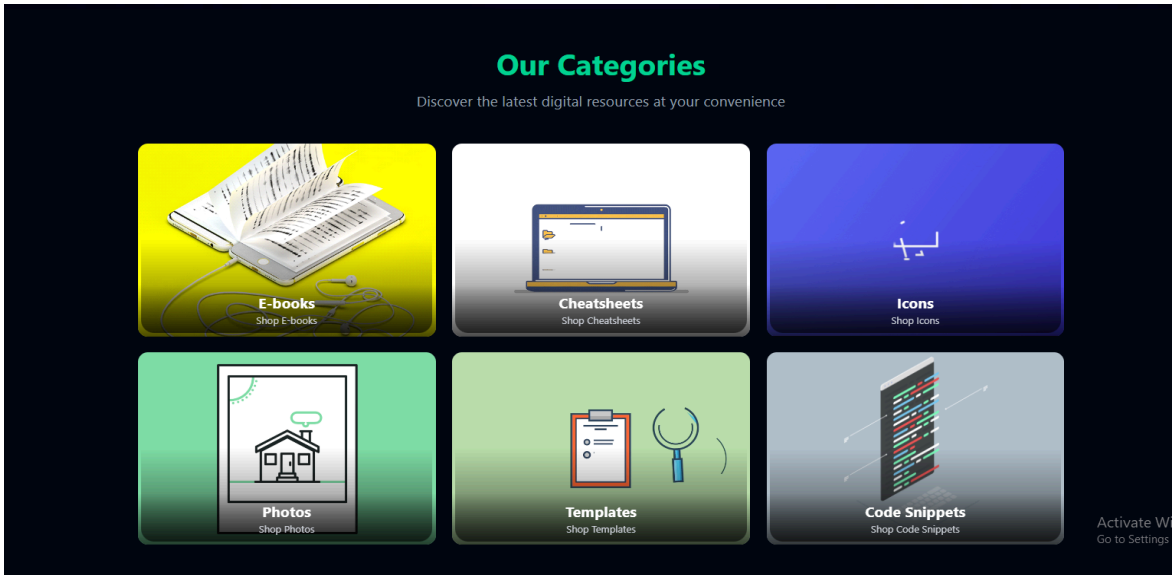


Backend Overview:

Created user model for users (Customer/Admin), product model for products and post model for community. Similarly created routes and controllers for them. Also added middleware (protectRoute - login and adminRoute - only admin can login to admin dashboard.

Created and connected to the database successfully. And the backend has been tested by using Postman. Inside the lib folder there are db and cloudinary connections and the server.js file is handling all the api calls. Also api key, secret and uri has been stored into .env file

Feature 1: Product Management:



Created product model which includes: name, description, price, image, category. When pressed in a category card, it will show similar products that fall in the same category.

ID: 22299480

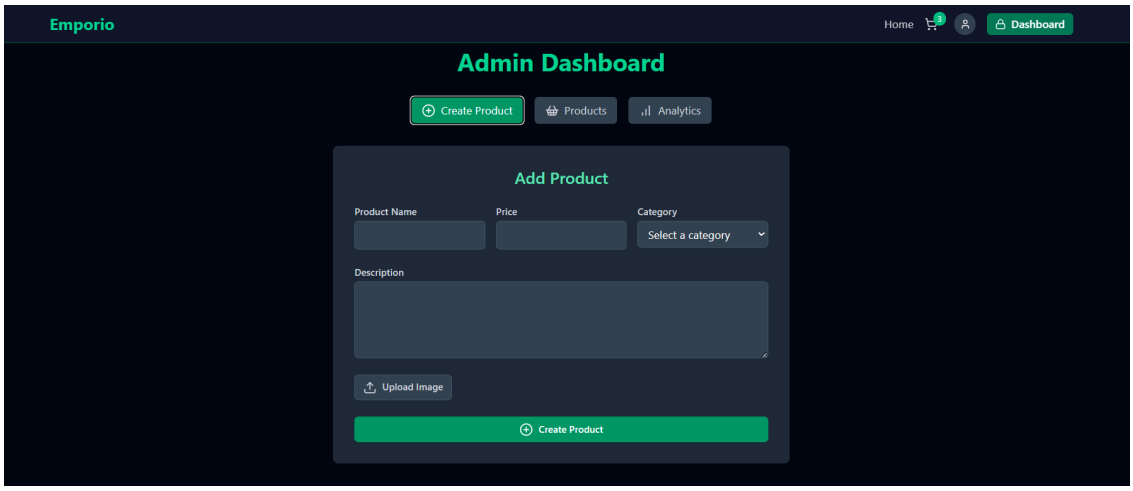
ProductModel

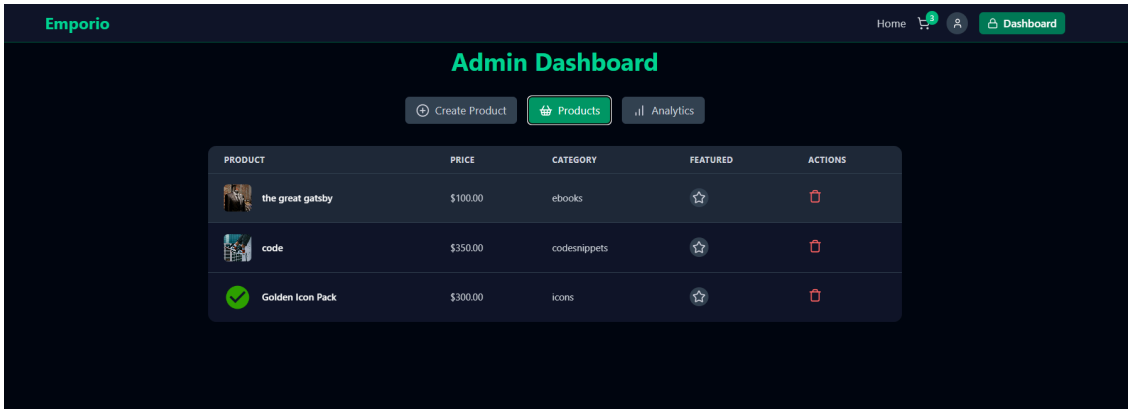
```
import mongoose from "mongoose";

const productSchema = new mongoose.Schema(
  {
    name: {…
    },
    description: {…
    },
    price: {…
    },
    image: {…
    },
    category: {…
    },
    // todo -> tag
    isFeatured: {…
    },
  },
  { timestamps: true }
);

const Product = mongoose.model("Product", productSchema);

export default Product;
```

productRoute

```
import express from "express";
import { adminRoute, protectRoute } from "../middleware/auth.middleware.js";
import {
  createProduct,
  deleteProduct,
  getAllProducts,
  getFeaturedProducts,
  getProductsByCategory,
  getRecommendedProduct,
  toggleFeaturedProduct,
} from "../controllers/product.controller.js";

const router = express.Router();

// gets all the product from db
router.get("/", protectRoute, adminRoute, getAllProducts); // only admin can access this
router.get("/featured", getFeaturedProducts); // everyone can access this
router.get("/category/:category", getProductsByCategory); // everyone can access this
router.get("/recommendations", getRecommendedProduct); // everyone can access this
router.post("/", protectRoute, adminRoute, createProduct); // only admin can access this
router.put("/:id", protectRoute, adminRoute, toggleFeaturedProduct); // only admin can access this
router.delete("/:id", protectRoute, adminRoute, deleteProduct); // only admin can access this

export default router;
```

Feature 2: Admin Dashboard:

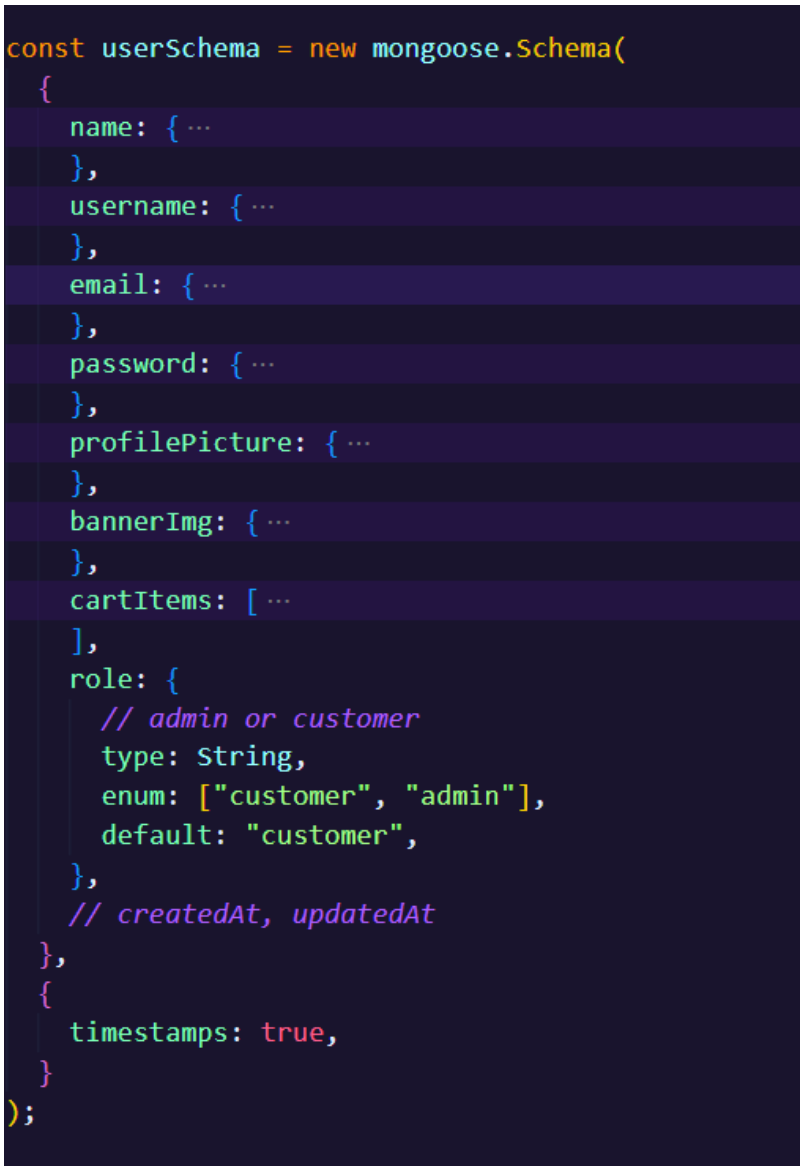




Admin can add a new product and also see the list and delete a product from the database. Here, the image is connected and also being uploaded to cloudinary along with the mongodb database.

ID: 22299480

User model



Sprint 2

Completed Features **(4)**

- **Search Functionality** (search by product's name, description, category)
- **Profile page** (both customers and admin can edit their name and picture)
- **Cart system** (Users can add products to the cart and also can increase or decrease the quantity of each product)
- **User Reviews and Ratings** (Users can review a product and also leave rating)

`***Quality of life update:** The signup and login ui has also been revamped.*`

Backend: (newly updated)

user.controller.js:

- Updates user profiles (name, username, email).
- Uploads profilePicture and bannerImg to Cloudinary.
- Without authentication, users can't enter the website, profile or cart.

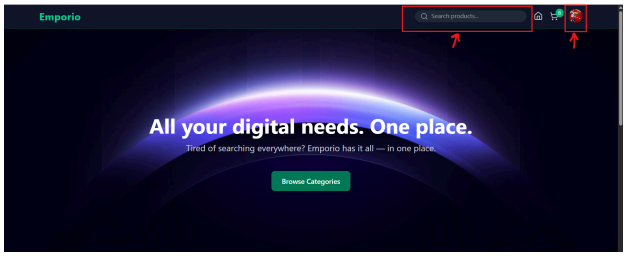
search.controller.js:

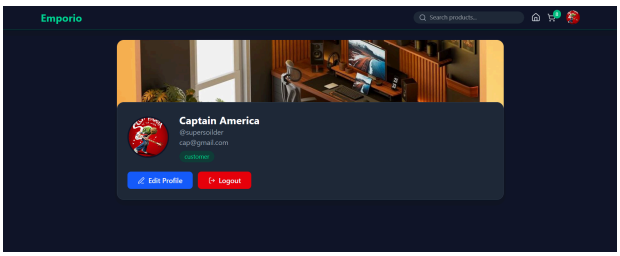
- Searches products by name, description, or category using regex (case-insensitive).
- Returns matched products or errors if the query is missing.

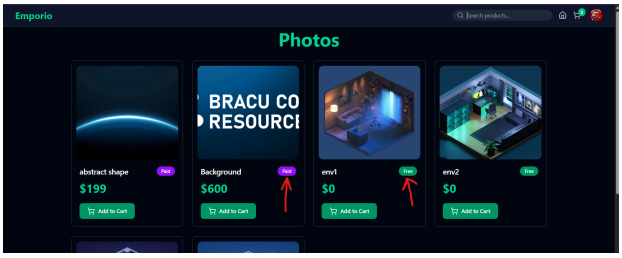
cart.controller.js:

- Manages cart operations for users.
- getCartProducts: Fetches products in the user's cart with quantities.
- addToCart: Adds a product or increments quantity if already in cart.
- removeAllFromCart: Clears the cart or removes a specific product.
- updateQuantity: Updates product quantity; removes product if quantity is zero.

ID: 22299480







backend > routes > user.route.js > ...

```
1  import express from "express";
2  import { protectRoute } from "../middleware/auth.middleware.js";
3  import { updateProfile } from "../controllers/user.controller.js";
4
5  const router = express.Router();
6
7  router.get("/profile", protectRoute, (req, res) => {
8    res.json(req.user);
9  });
10
11 // PUT update profile
12 router.put("/profile", protectRoute, updateProfile);
13 router.put("/update", protectRoute, updateProfile);
14
15 export default router;
16
```

user.route.js & **search.route.js**:

Added update profile to edit name and picture of users. They can directly add their profile and banner image and also they can search products

backend > models > product.model.js > productSchema

```
import mongoose from "mongoose";

const productSchema = new mongoose.Schema(
  {
    name: { ···
    },
    description: { ···
    },
    price: { ···
    },
    image: { ···
    },
    category: { ···
    },
    // todo -> tag
    isFeatured: { ···
    },
    isFree: { ···
    },
  },
  { timestamps: true }
);

// free -> price 0
productSchema.pre("save", function (next) {
  if (this.isFree) {
    this.price = 0;
  }
  next();
});

const Product = mongoose.model("Product", productSchema);

export default Product;
```

search.route.js:

backend > routes > search.route.js > ...

```
import express from "express";
import { searchProducts } from "../controllers/search.controller.js";

const router = express.Router();

router.get("/", searchProducts);

export default router;
```

This is the product.model.js:
Here all the fields for products are provided and the isFree has been added to it for free and paid tier products along with frontend changes (check box).

search.controller.js:

search.controller.js | user.controller.js | search.route.js | user.route.js

backend > controllers > search.controller.js > searchProducts

```
import Product from "../models/product.model.js";

export const searchProducts = async (req, res) => {
  try {
    const { query } = req.query;

    if (!query || query.trim() === "") {
      return res.status(400).json({ message: "Search query is required" });
    }

    const products = await Product.find({
      $or: [
        { name: { $regex: query,   const query: any   // name
        { description: { $regex: query, $options: "i" } }, // description
        { category: { $regex: query, $options: "i" } }, // category
      ],
    });

    res.json(products);
  } catch (error) {
    console.error("Error in searchProducts:", error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
```

ID: 22299480

And to wrap it up server.js:

backend > server.js > ...

```
import express from "express";
import dotenv from "dotenv";
import cookieParser from "cookie-parser";

// routes
import authRoutes from "./routes/auth.route.js";
import userRoutes from "./routes/user.route.js";
import postRoutes from "./routes/post.route.js";
import productRoutes from "./routes/product.route.js";
import cartRoutes from "./routes/cart.route.js";
import searchRoutes from "./routes/search.route.js";
import { connectDB } from "./lib/db.js";

dotenv.config();
console.log("MONGO_URI:", process.env.MONGO_URI);

const app = express();
const PORT = process.env.PORT || 5000;

app.use(express.json({ limit: "10mb" })); // allow to parse the body of the request
app.use(cookieParser());

app.use("/api/auth", authRoutes);
app.use("/api/post", postRoutes);
app.use("/api/products", productRoutes);
app.use("/api/cart", cartRoutes);
app.use("/api/user", userRoutes); // "/api/user/update"
app.use("/api/search", searchRoutes);

app.listen(5000, () => {
  console.log("SERVER is running on http://localhost:" + PORT);

  connectDB();
});

```

These three are newly added to server.js:

- app.use("/api/cart", cartRoutes);
- app.use("/api/user", userRoutes);
- app.use("/api/search", searchRoutes);

These routes serve as the backend endpoints that let the frontend perform important actions (cart management, profile updates, and product searching)

Sprint 3

Completed Features **(4)**

- **Announcement**: Admin can announce something and also pin the announcement which users will be able to see in the community page
- **Wishlist** via voting: Customers can vote for multiple products and admin will see it in admin dashboard's create product tab
- **Community** - discussion: Users (Admin and Customer) can discuss (chat) about something in the community page
- **Analytics**: This shows the total number of users, products, posts, and announcements. Only the admin can see this information.

`***Quality of life update:*** ***Most of the UI design has been revamped to make it smoother.***`

Backend:

announcements.controller.js:

- CRUD operation for announcements - Admins can manage site announcements
- Count total announcements - For analytics dashboard
- Handle pinned announcements - Important ones show first

post.controller.js:

- CRUD operation for community feed and discussion
- Show author info - Username and profile picture with each post

wishlist.controller.js:

- Add, view, remove wishlist items - Users can request products they want
- Vote on wishlist items - Popular requests will be uploaded by the admin
- Separate from regular products - Wishlist items don't appear in store

analytics.controller.js:

- Get platform statistics - Total users, products, posts, announcements
- Provide data for admin dashboard - real time data

ID: 22299480

backend > models > post.model.js > postSchema

```js
import mongoose from "mongoose";

const postSchema = new mongoose.Schema(
  {
    author: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    content: { type: String },
    image: { type: String },
    upvotes: [{ type: mongoose.Schema.Types.ObjectId, ref: "User" }],
    comments: [
      {
        content: { type: String },
        user: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
        createdAt: { type: Date, default: Date.now },
      },
    ],
  },
  { timestamps: true }
);

const Post = mongoose.model("Post", postSchema);

export default Post;

```

backend > routes > post.route.js > ...

```js
import express from "express";
import { protectRoute, adminRoute } from "../middleware/auth.middleware.js";
import {
  getFeedPosts,
  createPost,
  deletePost,
  getPostsCount,
  getPostAnalytics,
} from "../controllers/post.controller.js";

const router = express.Router();

// Get all posts
router.get("/", getFeedPosts);

// Create a post (protected)
router.post("/", protectRoute, createPost);

// Delete a post (protected)
router.delete("/:id", protectRoute, deletePost);

// Analytics endpoints (admin only)
router.get("/count", protectRoute, adminRoute, getPostsCount);
router.get("/analytics", protectRoute, adminRoute, getPostAnalytics);

export default router;

```

backend > routes > analytics.route.js > ...

```js
import express from "express";
import { protectRoute, adminRoute } from "../middleware/auth.middleware.js";
import { getAnalytics } from "../controllers/analytics.controller.js";

const router = express.Router();

router.get("/", protectRoute, adminRoute, getAnalytics);

export default router;

```

```js
import express from "express";
import {
  listAnnouncements,
  createAnnouncement,
  deleteAnnouncement,
  getAnnouncementsCount,
  getAnnouncementAnalytics,
} from "../controllers/announcement.controller.js";
import { protectRoute, adminRoute } from "../middleware/auth.middleware.js";

const router = express.Router();

// Public read
router.get("/", listAnnouncements);

// Admin-only create and delete
router.post("/", protectRoute, adminRoute, createAnnouncement);
router.delete("/:id", protectRoute, adminRoute, deleteAnnouncement);

// Analytics endpoints (admin only)
router.get("/count", protectRoute, adminRoute, getAnnouncementsCount);
router.get("/analytics", protectRoute, adminRoute, getAnnouncementAnalytics);

export default router;
```

backend > controllers > announcement.controller.js > ...

```js
import Announcement from "../models/announcement.model.js";

// Get all announcements (public)
export const listAnnouncements = async (req, res) => {
};

// Create new announcement (admin only)
export const createAnnouncement = async (req, res) => {
};

// Delete announcement (admin only)
export const deleteAnnouncement = async (req, res) => {
};

// Get announcements count for analytics
export const getAnnouncementsCount = async (req, res) => {
};

// Get announcement analytics
export const getAnnouncementAnalytics = async (req, res) => {
};

```